

Fuzzing Quantum Simulators: Develop a targeted fuzzer for quantum simulators to uncover inconsistencies in quantum circuit simulation

Project Idea 1: Oracle-Guided Fuzzing of Measurement Outcomes

Idea: Implement a fuzzer that generates quantum circuits and analyses their measurement distributions.

Goal: Find inputs for which the simulators diverge statistically (e.g. when sampling from the same circuit multiple times).

Learning outcomes:

- Statistical reasoning about quantum outputs
 - Implementation of oracle mechanisms
 - Comparison of simulation backends
-

Project Idea 2: Consistency Fuzzing of Statevectors

Idea: Run the same circuit across two simulators (e.g., Qiskit and PennyLane) and compute the fidelity between the resulting statevectors.

Goal: Identify cases where the statevectors deviate beyond a threshold.

Learning outcomes:

- Exposure to linear algebra in quantum states
 - Quantitative validation methods
 - Building reproducible test frameworks
-

Reading list:

- Introduction to Quantum Computing: <https://arxiv.org/pdf/quant-ph/0011013>
- Qiskit Textbook (Official tutorial): <https://learn.qiskit.org/course/introduction>
- Learn quantum programming – PennyLane: <https://pennylane.ai/qml>
- Cirq | Google Quantum AI: <https://quantumai.google/cirq>

Research papers:

- FuzzQ: https://kclpure.kcl.ac.uk/ws/portalfiles/portal/348929948/Authors-version_Shaking_Up_Quantum_Simulators_with_Fuzzing_and_Rigour_OOPSLA_25.pdf
- MorphQ: <https://arxiv.org/pdf/2206.01111>
- QDiff: <https://web.cs.ucla.edu/~miryung/Publications/ase2021-qdiff.pdf>
 - QuteFuzz: <https://popl25.sigplan.org/details/planqc-2025-papers/12/QuteFuzz-Fuzzing-quantum-compilers-using-randomly-generated-circuits-with-control-fl>

Applying echolocation more broadly

Project Idea 3: Cross-Architecture Echolocation Profiler

- **Title:** "Echo-Mapping the Memory Hierarchy: A Cross-Platform Persistence Profiler"
 - **High-Level Goal:** Develop a portable C/C++ framework to implement and apply the memory echolocation technique across different hardware architectures (e.g., modern x86, ARM, RISC-V).
 - **Key Tasks:**
 1. **Implement the Probe:** Write a portable timing probe using architecture-specific high-resolution timers (rdtscp for x86, cntvct_el0 for ARMv8, rdcycle for RISC-V).
 2. **Calibration:** Run the probe on at least two different platforms (e.g., a lab x86 machine and a Raspberry Pi 4/5 for ARM) to collect millions of latency data points.
 3. **Pattern Analysis & Visualisation:** Use a data analysis tool (like Python with Pandas/Matplotlib/Scikit-learn) to automatically identify and visualise the latency clusters corresponding to different memory hierarchy levels on each platform.
 4. **Comparison:** Create a comparative report and visualisations (like side-by-side violin plots or histograms) showing how the memory hierarchy "maps" (i.e., the latency, size, and number of levels) differ between the tested architectures.
 - **Deliverables:** A portable C/C++ probing tool, Python analysis scripts, and a report comparing the memory hierarchy profiles of different architectures.
 - **The project** combines systems programming (C/C++, inline assembly), data collection, and data science/visualisation (Python), hitting many core CS skills.
-

Project Idea 4: Automated Latency Cluster Detection Framework

- **Title:** "Auto-Echo: A Framework for Automated Discovery of Memory Hierarchy Latency Patterns"
- **High-Level Goal:** Build a framework that not only collects echolocation data but also *automatically* discovers the number and boundaries of memory hierarchy levels without prior knowledge.
- **Key Tasks:**
 1. **Data Collection:** Implement or reuse the echolocation probe to gather a large dataset of memory access latencies from a target machine.
 2. **Noise Filtering:** Implement basic statistical filters (e.g., removing extreme outliers, using a moving average) to clean the raw timing data.

3. **Unsupervised Learning:** Apply unsupervised machine learning algorithms (e.g., K-Means Clustering, Gaussian Mixture Models (GMM), or DBSCAN) to the latency data to automatically identify distinct clusters.
 4. **Model Selection & Reporting:** Implement a method (like the Elbow Method or Silhouette Score for K-Means) to automatically determine the most likely number of memory hierarchy levels (k clusters). The framework should output the discovered number of levels and the latency range for each.
 5. **Validation:** Compare the framework's automated findings against the known specifications of the test machine (e.g., does it correctly identify L1, L2, L3, and DRAM as four distinct clusters?).
- **Deliverables:** A Python-based framework that takes latency data as input and outputs a report on the discovered memory hierarchy structure. A report validating the framework's accuracy on one or two known machines.
 - **The project** is a great blend of systems and machine learning. It directly tackles the "pattern spotting" aspect mentioned in the paper below and provides a concrete application of classic unsupervised learning algorithms to solve a real-world systems problem. It moves beyond manual inspection of plots towards automation.

Reading list:

- Shouting at Memory: Where Did My Write Go? <https://drops.dagstuhl.de/storage/00lipics/lipics-vol333-ecoop2025/LIPIcs.ECOOP.2025.41/LIPIcs.ECOOP.2025.41.pdf>
- Artifact for paper "Shouting at Memory: Where Did My Write Go?" <https://zenodo.org/records/15045528>